

A three-panel workspace for prompt-driven VTK

VTK Prompt UI

Recents

create a cube

create a sphere

show me hedgehog glyphs on a ...

show me a red tetrahedron

show me a tetrahedron colored r...

using mop tools, show me the ap...

using the mop tools, show me th...

Find a VTK example that does str...

list the mop tools

tell me the rendering classes. us...

tell me about the api for vtkStrea...

create a red sphere

show me a visualization of the e...

show me an example for exodus ...

create a cube

create a cfd visualization

create a superquadric

show me how to visualize the co...

create a cow

Generated Code

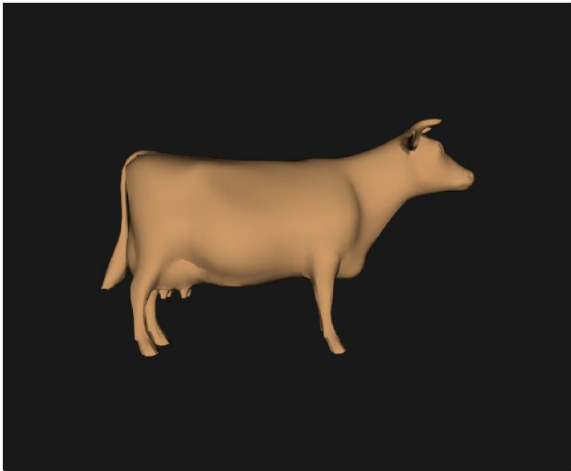
```
1 # renderer is a vtkRenderer injected by this webapp
2 # use your own vtkRenderer in your application
3 import vtk
4
5 # Create a reader for the cow dataset in BYU format
6 reader = vtk.vtkBYURenderer()
7 reader.SetFileName('cow.g')
8 reader.Update()
9
10 # Smooth the polygonal data
11 smoother = vtk.vtkSmoothPolyDataFilter()
12 smoother.SetInputConnection(reader.GetOutputPort())
13 smoother.SetNumberOfIterations(10)
14 smoother.SetRelaxationFactor(0.3)
15
16 # Compute surface normals
17 normals = vtk.vtkPolyDataNormals()
18 normals.SetInputConnection(smoother.GetOutputPort())
19
20 # Map the smoothed polygonal data to graphical primitives
21 mapper = vtk.vtkPolyDataMapper()
22 mapper.SetInputConnection(normals.GetOutputPort())
23
24 # Create an actor to represent the cow model
25 actor = vtk.vtkActor()
26 actor.SetMapper(mapper)
27 actor.GetProperty().SetColor(0.8, 0.6, 0.4) # Set color to a light brown
28
29 # Add the actor to the renderer
30 renderer.AddActor(actor)
```

192.168.50.157:11434/v1/qwen2.5-coder-32b-ctb16k

Describe VTK visualization

e.g., Create a red sphere with lighting

VTK Visualization



CONVERSATION

CONSOLE

show me how to visualize the cow dataset

This code loads the cow dataset, which is a common 3D model used in computer graphics and visualization. It creates a vtkPolyDataReader to read the dataset from a file named 'cow.g'. A vtkPolyDataMapper is then used to map the polygonal data to graphical primitives that can be rendered. An actor is created to represent this data, which includes properties such as color and shading. Finally, the actor is added to the provided renderer object so it can be visualized.

Read the file cow.g with vtkBYURenderer and render it as a surface

This code loads the cow dataset using the vtkBYURenderer, which is specifically designed to read files in the BYU format. A vtkPolyDataMapper is used to map the polygonal data from the reader to graphical primitives that can be rendered. An actor is created to represent this data, with properties set for color and shading. Finally, the actor is added to the provided renderer object so it can be visualized as a surface.

RECENTS

Pinned + past conversations

GENERATED CODE

Editor · undo/redo · Run · data chip

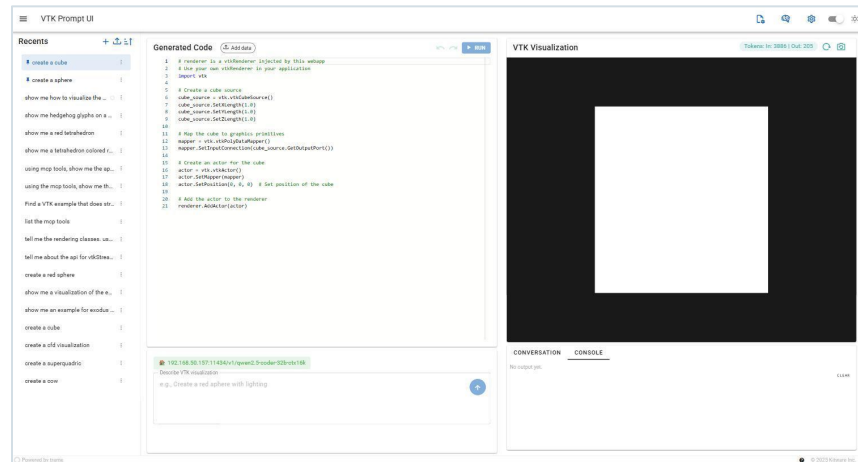
VISUALIZATION + TABS

Live render · Conversation / Console

Concurrent Conversations

1 Each conversation owns its state

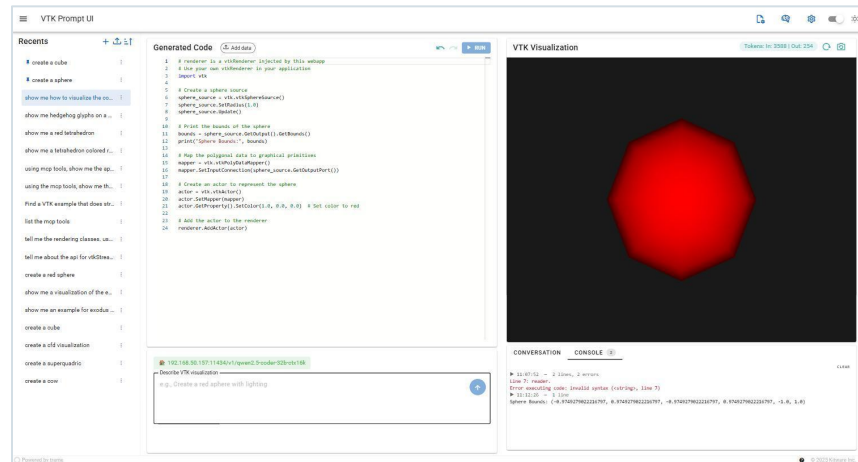
- Prompt client is now stateless: each conversation carries its own messages, code, and scene.
- Conversations generate at the same time; a result is delivered to the conversation that started it, not whatever you are viewing.
- The drawer shows which conversation is busy; switching back to one still generating no longer shows a stale result.
- Generated code can no longer pop up a native window or corrupt the app's render window.



Per-Conversation Console

2 Everything a run produces, in one place

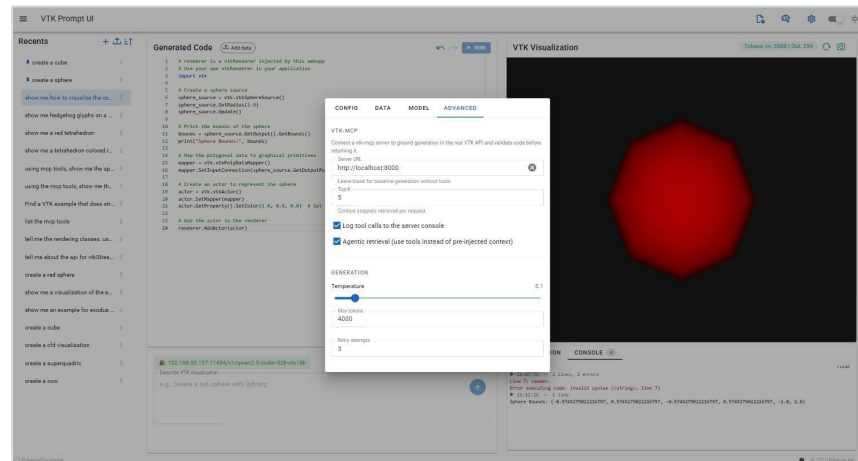
- stdout / stderr from generated code now surface in a Console tab beside the transcript — a print() is no longer lost to the server terminal.
- Lines are classified by stream, not text: a printed class name like vtkErrorCode is output, not an error.
- The console is the single record of a run — errors, warnings, and the resolver's suggestions all land here; the floating alert is gone.
- A severity-colored tab badge flags failures; the console is stored per conversation.



MCP Tooling for Local Models

3 Tool calls that work with local backends

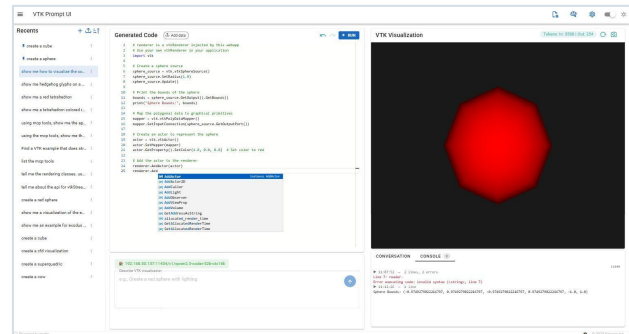
- Runs tool calls that local backends emit as inline text — for quantized / self-hosted models without native tool calling.
- The model can list and call `vtk-mcp` tools itself, then use the results to write the visualization code.
- Optional logging of every `vtk-mcp` tool call, as a persisted setting, so tool use is inspectable.
- Agentic-retrieval toggle: skip pre-injected context and let the model drive retrieval on its own.



Authoring Experience

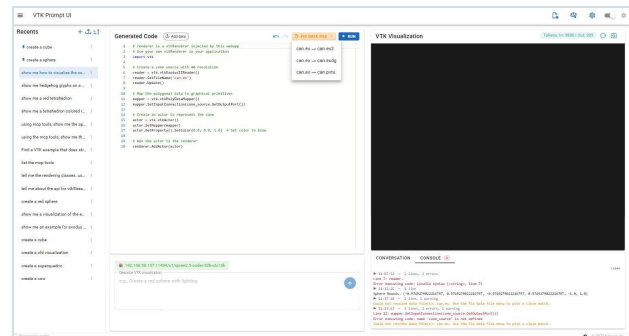
4 Monaco autocomplete & hover

- jedi-backed completion and hover, in-process over the existing websocket — no extra server.
- Resolves vtk with real docstrings, plus the objects injected into generated code (renderer, render_window).
- Completions stay stable while typing; the editor only remounts on a conversation switch.



5 Data files grounded in VTK data

- Indexes the VTK data collection (VTK_PROMPT_DATA_ROOT) and uploaded files; can fetch known datasets on demand.
- An unresolved name gets the closest matches (exact stem, then fuzzy) via a "Fix data file" menu that swaps and re-runs.



"Fix data file": can.ex → can.ex2 / can.exdg / can.pvtu

Adding a thinking stage

6

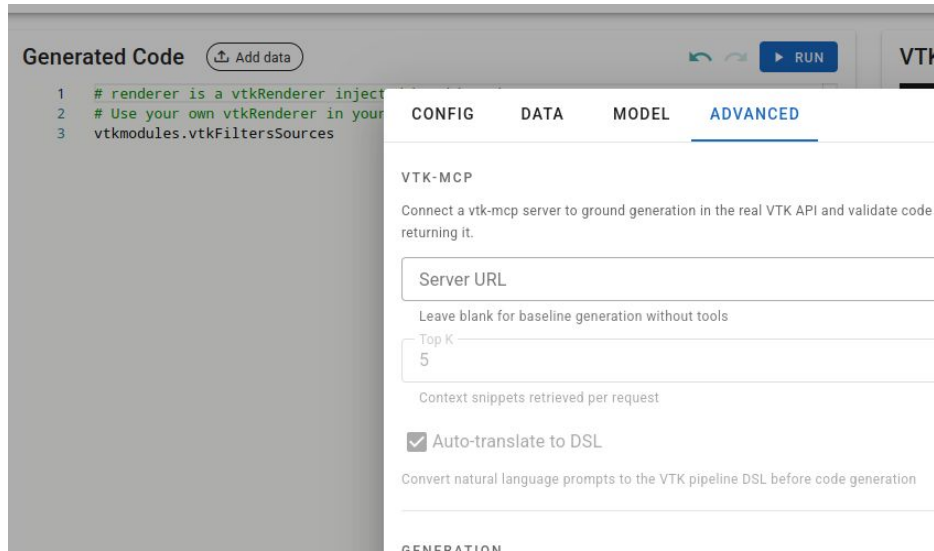
Thinking stage translates a natural language query into a DSL like language which tries to understand the user's intention

- It parses request's intent and transforms it as a structured pipeline specification, naming the filters, parameters, and outputs involved. This grounds the rest of the pipeline in a precise, unambiguous intermediate form instead of raw text, improving retrieval accuracy and reducing hallucinated API calls.
- It is enabled by default, with a toggle in the UI settings to opt out when raw natural language generation is preferred.

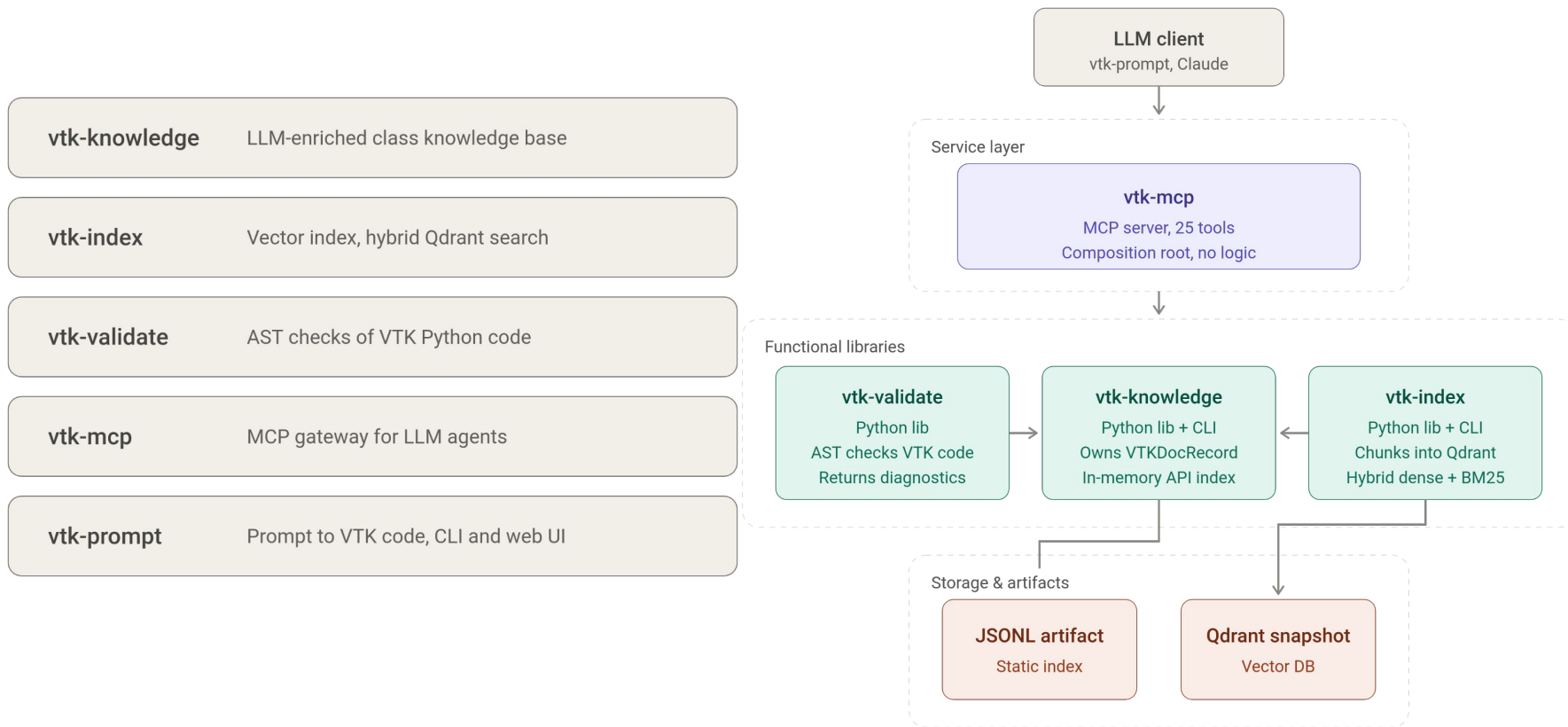
7

Aid debugging conversation with LLM and tool usage

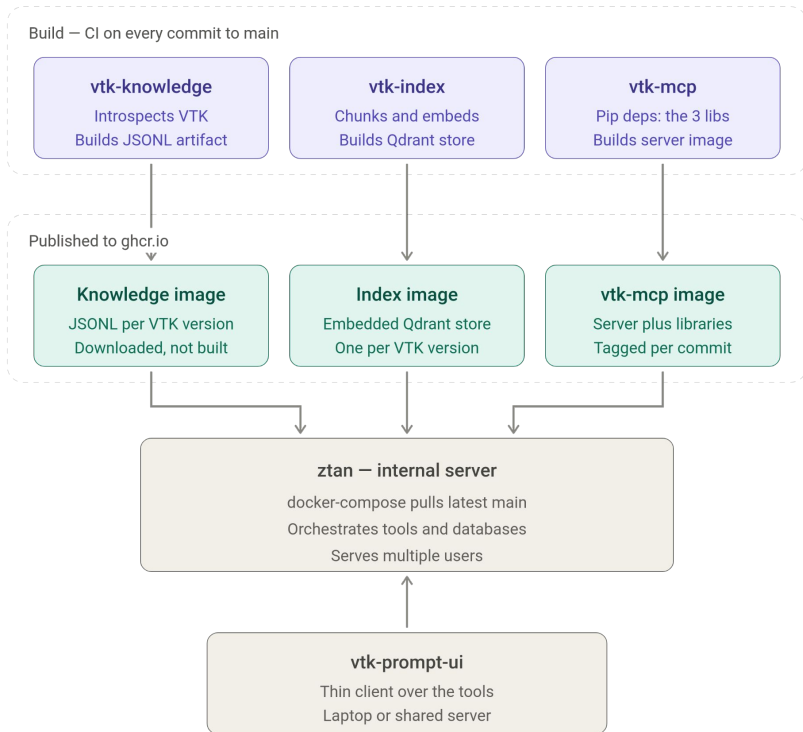
- For CLI and UI, a `--debug` flag dumps the full LLM conversation to stdout, including injected context and vtk-mcp tool calls and results



vtk-knowledge ecosystem



vtk-knowledge deployment



```
vtk-mcp:
  build: .
  environment:
    VTK_MCP_TRANSPORT: http
    VTK_MCP_HTTP_PORT: "8000"
    VTK_MCP_ENABLE_RETRIEVAL: "true"
    VTK_MCP_QDRANT_URL: http://qdrant:6333
    VTK_MCP_KNOWLEDGE_ARTIFACT_PATH: /app/data/vtk-knowledge.jsonl
  ports:
    - "8000:8000"
  depends_on:
    - qdrant
  volumes:
    - ./data:/app/data:ro

volumes:
  qdrant_data:
```

CONFIG

DATA

MODEL

ADVANCED

VTK-MCP

Connect a vtk-mcp server to ground generation in the real VTK API and validate code before returning it.

Server URL



Leave blank for baseline generation without tools

vtk-mcp functions

vtk-knowledge

17 tools

vtk_is_a_class()

vtk_get_class_synopsis()

vtk_get_class_methods()

vtk_search_classes()

vtk_get_class_action_phrase()

vtk_get_class_semantic_methods()

vtk_get_class_module()

vtk_get_class_role()

vtk_get_method_info()

vtk_get_module_classes()

vtk_get_class_input_datatype()

vtk_get_method_doc()

get_vtk_class_info_python()

vtk_get_class_output_datatype()

vtk_get_method_signature()

vtk_get_class_doc()

vtk_get_class_visibility()

vtk-validate

validate_vtk_code()

vtk_validate_import()

is_dsl_prompt()

translate_prompt_to_dsl()

vtk-index (Qdrant)

vector_search_docs()

vector_search_examples()

vtk-mcp itself

vtk_version_info()